

Java GUI Programming Formative Assessment

Using MagikLayout to investigate Java Swing layout managers

Student name	_____
Registration number	_____
Class and session	_____ Date: _____
Assessment	Individual formative assessment Duration: 60 minutes Total: 40 marks

Purpose. This assessment checks whether you can predict, build, test, and explain Java Swing layouts. You will record a prediction before using MagikLayout, compare it with the observed result, and explain the rule that caused the result.

Learning outcomes

- Select BorderLayout, FlowLayout, or GridLayout for a stated interface requirement.
- Predict how components move or resize when a container changes size.
- Use a nested JPanel to combine layout managers and prevent region conflicts.
- Diagnose a layout problem and justify a repair using generated Java evidence.

Assessment procedure

1. Complete the lecturer's Java Swing Layout Knowledge Pre-Test before opening MagikLayout.
2. Open <https://magik-layout.mhdazlan.cc> and complete Tasks 1 to 5 in this booklet.
3. Write your own prediction before testing each layout. A changed or incorrect prediction is acceptable when your explanation shows what you learned.
4. Show the requested checkpoints to the lecturer. Submit this booklet before receiving the post-test link.
5. Complete the post-test once, individually, without returning to MagikLayout or discussing answers.

Conditions

- You may use MagikLayout's Playground, Challenges, generated Java panel, and built-in hints when the task permits them.
- Do not use search engines, notes, messaging, or another student's screen during the assessment.
- The marks are formative: they identify the concepts that need further teaching. Follow your lecturer's instructions about whether the score contributes to coursework.

Task 1 BorderLayout prediction and observation 8 marks

Build. In the Playground, select BorderLayout. Add five components labelled Header, Navigation, Workspace, Tools, and Status. Place one component in each available region. Resize the frame wider, narrower, taller, and shorter.

Prompt	Your prediction before testing	What you observed in MagikLayout
Which component will receive most of the extra width and height?		
Which components keep a height close to their preferred height?		
Which components keep a width close to their preferred width?		

Explain the layout rule that caused the observed resizing behaviour.

Copy one relevant generated Java statement from the Code panel.

Lecturer checkpoint: BorderLayout shown and resized Initials: _____ Time: _____

Task 2 FlowLayout and GridLayout comparison 8 marks

Build. Reset the Playground. Add six buttons labelled A to F. Test the same components first with FlowLayout and then with GridLayout. For each manager, compare a wide frame with a narrow frame.

Feature	FlowLayout observation	GridLayout observation
Component size		
Order of components		
Effect of narrowing the frame		
Use of empty space		

For a row of compact buttons that may wrap, which manager would you choose and why?

For a keypad whose buttons must have equal-sized cells, which manager would you choose and why?

Task 3 Repair a BorderLayout region conflict 10 marks

Create the problem. Use BorderLayout. Add a Save button to SOUTH, then add a Cancel button directly to SOUTH. Record what happens and the rule shown by MagikLayout.

What happened to the first button, and why?

Repair the structure. Create a JPanel in SOUTH, give that panel a suitable layout, and place both buttons inside it. Resize the frame to confirm that both remain visible.

Evidence	Your record
Parent container and its layout	
Nested panel and its layout	
Children of the nested panel	
Two generated Java lines that prove the repair	

Explain why nesting fixes the problem instead of placing both buttons directly in SOUTH.

Lecturer checkpoint: Both buttons visible in the nested panel Initials: _____ Time: _____

Task 4 Graded MagikLayout challenges 8 marks

Complete two challenges. Choose one Reflow challenge and one Reverse challenge. Make one independent attempt before using a hint. Record the evidence below; your lecturer may choose the exact challenge titles.

Evidence	Challenge 1 Reflow	Challenge 2 Reverse
Challenge title		
First prediction or structure		
First check result	☐ Correct ☐ Incorrect	☐ Correct ☐ Incorrect
Hint used	☐ No ☐ Yes	☐ No ☐ Yes
Final check result	☐ Passed ☐ Not yet	☐ Passed ☐ Not yet
Swing rule learned		

Choose one mistake you made. Explain how the feedback changed your understanding.

Lecturer checkpoint: Challenge results viewed Initials: _____ Time: _____

Task 5 Transfer to a new interface 6 marks

Design brief. Plan a Student Registration window containing a title, a central area with Name, Programme, and Semester fields, and a bottom row with Submit and Clear buttons. Use only BorderLayout, FlowLayout, GridLayout, and nested JPanel containers.

A Draw the component tree or labelled wireframe. Show each JPanel, its layout manager, and the component placement.

B Justify your design. Explain why each layout manager suits its part of the interface and predict what will happen when the window becomes wider.

Submission and post-test

- ☐ I wrote my full name and registration number.
- ☐ I completed all five tasks and obtained the required lecturer checkpoints.
- ☐ I submitted this hard-copy booklet before opening the post-test.
- ☐ I completed the post-test once and submitted it successfully.

Student declaration: The responses in this booklet and the post-test are my own work.

Signature: _____ Date: _____

Post-use experience survey. This questionnaire is separate from the learning assessment. Complete it only according to the consent and research instructions provided by the lecturer; survey responses should not affect course marks.